

WEEK8

BUILDING AI AGENT TOOLS



LangChain



COURSES. CLARITY. CONFIDENCE.

12 Week Agenda

Week 1: Introduction to LangChain & AI Workflows

Week 2: Working with Prompts, Chains & Memory

Week 3: AI Resume Analyzer

Week 4: PDF Document Summarizer (RAG Project)

Week 5: AI Email Assistant

Week 6: YouTube Transcript Summarizer



12 Week Agenda

Week 7: Customer Support Chatbot

Week 8: Building AI Agent Tools – Part 1

Week 9: Building AI Agent Tools – Part 2

Week 10: Meeting Notes Generator

Week 11: Travel Itinerary Planner Agent

Week 12: Local Knowledge Assistant



WEEK 8

BUILDING AI AGENT TOOLS – PART 1



Learning Objectives

Understand the difference between a **Chatbot** and an **Agent**

Learn what **MCP (Model Context Protocol)** is and why it matters.

Understand the **Client-Server** architecture for AI.

Build: Your first "Hello World" MCP Server using Python.



Why do we need this?



The Problem:

AI Models (like GPT-4) are trapped in a text box. They can't *do* anything (check time, read files, query databases).



The Old Way:

Paste data into the chat manually. (Slow, boring).

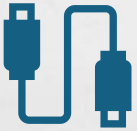


The Agent Way:

Give the AI "Hands" (Tools) to fetch data itself



Introducing MCP



Analogy:

"USB for AI".



Before MCP:

Every AI app (Claude, ChatGPT, Cursor) had a different way to connect tools.
Spaghetti code!



With MCP:

One standard open standard. Build a tool **once**, it works everywhere.



Open Source:

Created by Anthropic, now an open standard.

User Defined
Function

```
def get_user_orders(user_id):  
    ...
```

Without MCP

With MCP



Plain HTTPS vs MCP:

- **PLAIN HTTPS APPROACH:**

```
# Server has endpoint
@app.get("/api/add")
def add(a: int, b: int):
    return a + b
```

```
# You MUST hardcode the client to know this endpoint
exists
response =
requests.get("http://localhost:8000/api/add?a=10&b=20")
```

Problems:

- ✗ AI doesn't know what endpoints exist
- ✗ You must hardcode tool discovery
- ✗ Every server has different API structure
- ✗ AI can't automatically choose the right tool
- ✗ No standardization



MCP Approach:

```
# Server declares tools
@mcp.tool()
def add(a: int, b: int):
    """Add two numbers together."""
    return a + b

# Client AUTOMATICALLY discovers tools
tools = await session.list_tools() # Gets: [add_numbers, multiply_numbers, ...]

# LLM reads tool descriptions and picks the right one!
# "User asked 5*6, I see multiply_numbers tool, I'll call that"
```

Benefits:

- ✓ **Self-describing** - Tools are discoverable
- ✓ **AI-native** - LLMs can read descriptions and pick tools
- ✓ **Standardized** - Same protocol everywhere
- ✓ **Dynamic** - Add new tools without changing client code
- ✓ **Type-safe** - Type hints tell AI what data to send



Real-World Analogy:

HTTPS (MANUAL):

You: "Hello restaurant, I want to order"
Restaurant: "We have many dishes"
You: "Which ones?"
Restaurant: "I don't know, you figure it out"
✗ You have to guess or call each dish individually

MCP (AUTOMATIC):

You: "Hello restaurant"
Restaurant: *automatically shows you menu with descriptions*
You: "I want pasta"
Restaurant: *automatically processes it*
✓ You can see everything available and pick intelligently

MCP turns your functions into **discoverable capabilities** that **AI can reason about and use automatically**. HTTPS is just networking - MCP is how we make AI-native applications.

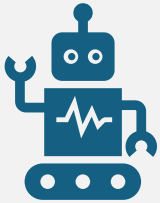


Key Points:

- **DISCOVERY** – "TOOLS TELL AI WHAT THEY CAN DO"
- **AI-READINESS** – "LLMS UNDERSTAND TOOL DESCRIPTIONS"
- **STANDARDIZATION** – "SAME MCP PROTOCOL WORKS EVERYWHERE"
- **SCALABILITY** – "ADD 100 TOOLS, AI STILL PICKS THE RIGHT ONE"
- **JSON-RPC** – "STANDARDIZED MESSAGE FORMAT"
- **TRANSPORT AGNOSTIC** – "WORKS WITH STDIO, HTTP, WEBSOCKETS, ETC."



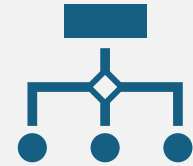
MCP Capabilities



TOOLS (WHAT WE'RE USING) –
FUNCTIONS AI CAN CALL



RESOURCES – READ-ONLY
DATA (FILES, URLS, DATABASES)



PROMPTS –
TEMPLATES/SYSTEM PROMPTS



MCP's goal: Help LLMs stay in context



Servers tell clients what they can do



Clients only request what's needed



LLMs only load relevant tools (saves tokens/cost)



Important for production (context windows matter)



Server Initialization Handshake

CLIENT: "HI, I'M AN MCP CLIENT"

SERVER: "HI, I SUPPORT PROTOCOL V2024-11-05"

CLIENT: "GREAT! WHAT TOOLS DO YOU HAVE?"

SERVER: "HERE ARE MY TOOLS WITH DESCRIPTIONS"

Stateless vs Stateful

- **Stateless servers** (better) - Each tool call independent
- **Stateful servers** (our Week 8 chatbot) - Remember conversation history

For your 3 servers: **Stateless** (each call independent)



Three main methods of running MCP servers

1. [MCP.RUN\(\)](#) (BUILT-IN FASTMCP METHOD)

Pros:

- ✓ Simple, one-line to start
- ✓ Handles everything internally
- ✓ Good for quick demos

Cons:

- ✗ Limited control over port/host
- ✗ Can't customize server behavior
- ✗ Hard to run multiple servers (all try port 8000)
- ✗ No logging control

```
mcp = FastMCP("My Server")

@mcp.tool()
def my_tool():
    pass

mcp.run(transport="sse") # or "stdio"
```



2. uvicorn.run() (Direct ASGI Server) – What we're using

```
import uvicorn
```

```
mcp = FastMCP("MY SERVER")
```

```
app = mcp._server.app
```

```
uvicorn.run(app, host="127.0.0.1", port=8000,  
            log_level="info")
```

Pros:

- ✓ Full control over port (8000, 8001, 8002, etc.)
- ✓ Multiple servers on different ports
- ✓ Customize logging, workers, reload behavior
- ✓ Production-ready
- ✓ Can add middleware, CORS, etc.

Cons:

- ✗ Slightly more code
- ✗ Need to access internal `._server.app`



3. stdio Transport (Subprocess)

Pros:

- ✓ No port management needed
- ✓ Lightweight (no HTTP overhead)
- ✓ Client controls server lifecycle

Cons:

- ✗ Only one client per server instance
- ✗ Can't have multiple independent servers running
- ✗ Not suitable for production
- ✗ Limited to local machine

```
from mcp.client.stdio import stdio_client
```

```
server_params = StdioServerParameters(  
    command="python",  
    args=["server.py"]  
)
```

```
async with stdio_client(server_params) as (read, write):  
    # Client spawns server as subprocess  
    # Communication via stdin/stdout pipes
```



Comparison Table:

Feature	<u>mcp.run()</u>	<u>uvicorn.run()</u>	<u>stdio</u>
Multiple Servers	✗ No (port conflict)	✓ Yes (different ports)	✗ No
Control over Port	✗ Limited	✓ Full	✗ N/A
Production Ready	⚠ Partial	✓ Yes	✗ No
Multiple Clients	⚠ Single	✓ Multiple	✗ Single
HTTP/Network	✓ Yes	✓ Yes	✗ No (pipes)
Easy Setup	✓ Yes	⚠ Medium	✓ Yes
Customization	✗ Limited	✓ Full	⚠ Medium



Client-Server Mode | Session: Claude & Python via MCP



Client-Server Model

The Server (You build this):

A script that has functions (``add_numbers``, ``get_weather``). It sits and waits.

The Client (The AI):

It connects to your server. It sees the list of tools. It decides *when* to call them.



Understanding MCP Protocol Stack:

JSON-RPC 2.0 = THE MESSAGE FORMAT

- MCP USES **JSON-RPC 2.0** FOR THE ACTUAL MESSAGES/COMMANDS. EXAMPLE:

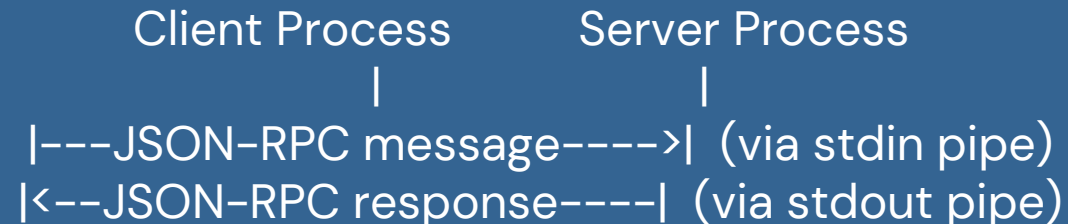
```
{  
  "jsonrpc": "2.0",  
  "method": "tools/call",  
  "params": {  
    "name": "add_numbers",  
    "arguments": {"a": 10, "b": 20}  
  },  
  "id": 1  
}
```

Transport = How Messages Are Delivered

STDIO AND HTTP/SSE ARE JUST TRANSPORT LAYERS THAT CARRY JSON-RPC MESSAGES:

1. STDIO (STANDARD INPUT/OUTPUT)

- JSON-RPC MESSAGES TRAVEL THROUGH **PROCESS PIPES** (STDIN/STDOUT)
- CLIENT SPAWNS SERVER AS SUBPROCESS
- MESSAGES GO: CLIENT STDOUT → SERVER STDIN AND SERVER STDOUT → CLIENT STDIN
- **ANALOGY:** TALKING TO SOMEONE SITTING NEXT TO YOU (LOCAL, DIRECT)



Summary

- **JSON-RPC** = THE LANGUAGE/FORMAT (WHAT YOU SAY)
- **STDIO/HTTP** = THE DELIVERY METHOD (HOW YOU SEND IT)

If Multiple Servers

**WITH STDIO, YOU CAN'T HAVE 3 SERVERS RUNNING SIMULTANEOUSLY.
WITH HTTP:**

- ☒ SERVER 1 ON PORT 8000 (ADD_NUMBERS)
- ☒ SERVER 2 ON PORT 8001 (MULTIPLY_NUMBERS)
- ☒ SERVER 3 ON PORT 8002 (GET_CURRENT_TIME)
- ☒ CLIENT CONNECTS TO ALL 3 AT ONCE
- ☒ LLM PICKS WHICH SERVER TO CALL

How we Code It



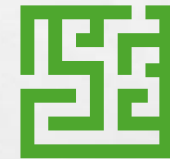
The Function:

Normal Python code.



The Decorator (`@mcp.tool`):

The "For Sale" sign. Tells the server to expose this function.



The Docstring:

The "Prompt". Tells the AI **what** this tool does.

```
```python
@mcp.tool()
def add_numbers(a: int, b: int):
 """Adds two numbers"""
 return a + b
```
```



WEEK 9: BUILDING THE "WEBSITE AGENT"



BUILDING AI AGENT TOOLS (PART 2)

– ADVANCED RAG & STATEFUL AGENTS



Learning Objectives

Combine Skills: RAG (Vector DBs) + MCP (Server).

Understand State: How to give tools "memory" (ChromaDB).

Build: A "Website Knowledge" Server that crawls URLs on demand.

Demo: Connect it to a real AI (Claude) and watch the magic.



The "Brain" and The "Mouth"

The Brain (Backend):

- LangChain + ChromaDB. Handles the heavy lifting (crawling, embedding).

The Mouth (Frontend/API):

- MCP Server. Exposes two simple buttons to the AI
 - 1. ``learn_website(url)``
 - 2. ``ask_question(query)``



Tools with Memory

Stateful:

- Step 1: "Learn React Docs" → System **Changes** (Database updates).
- Step 2: "How do hooks work?" → System uses the **Updated** Database.

Persist Directory:

- Why we save ChromaDB to disk.



Integration Demo

Scenario:

User: "Hey Claude, read the docs for 'FastMCP' and write me a tutorial."

Behind the Scenes:

1. Claude sees `learn\_website` tool.
2. Claude calls it.
3. **Your Server** crawls & indexes.
4. Claude sees "Success".
5. Claude calls `ask\_question` to verify details.
6. Claude writes the tutorial.



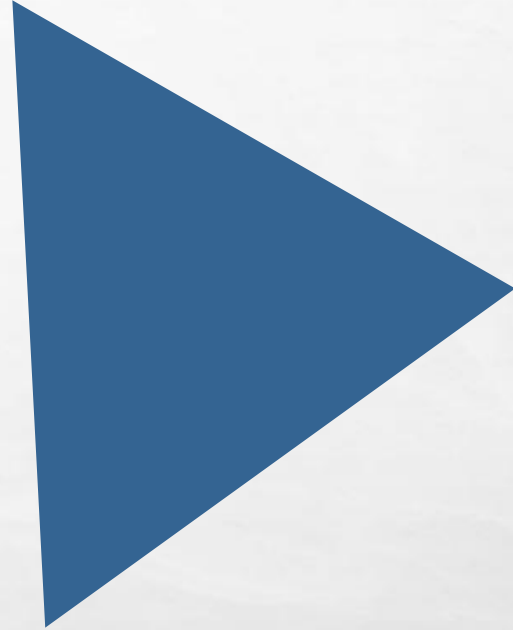
Today's Architecture



Why LangSmith?



CODE WALKTHROUGH



THANK YOU!

